CESI
ÉCOLE D'INGÉNIEURS

Développement d'applications distribuées

CER | Sécurité

Vendredi 12 juin 2026

Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

AAV **2**

Étude de cas : Sécurité **3**
- Mots clés . . . 3
- Contexte . . . 3
- Problématique . . . 3
- Contraintes . . . 3
- Généralisation . . . 3
- Livrables . . . 4
- Pistes de solutions . . . 5
- Plan d'actions . . . 5

Définition : Mots clés **6**
- CORS . . . 6
- React Context . . . 6
- Provider . . . 7
- Access store . . . 7
- Hooks . . . 8
- Loadbalancing . . . 8

Analyse et résolution : Sécurité **10**
- 1. Stockage du jeton (XSS / CSRF) . . . 10
- 2. Centraliser dans un access store . . . 10
- 3. Hooks et composants réutilisables . . . 12
- 4. Transport du jeton et CORS . . . 12
- 5. Routage par loadbalancing . . . 14
- 6. Architecture retenue . . . 14
- Conclusion . . . 15

Capitalisation **16**

Ressources **17**

AAV

---

6 Développer des composants réutilisables

3 Employer un access store

5 Interagir avec les composants (hooks)

5 Utiliser (Modéliser) JWT pour le processus d'authentification

4 Utiliser le système de loadbalancing pour réaliser le routage

ÉTUDE DE CAS : SÉCURITÉ

MOTS CLÉS

- CORS
- React context
- Provider
- Access store
- Hooks
- Loadbalancing

CONTEXTE

Yanis a fini de dev la partie token de son backend, cependant un problème persiste. Il se pose donc des questions sur la sécurité frontend de son site web.

PROBLÉMATIQUE

Comment sécuriser son token coté frontend ?

CONTRAINTES

- JWT
- React
- Problèmes de CORS

GÉNÉRALISATION

- Sécurité
- Gestion des secrets

LIVRABLES

- Code frontend sécurisé avec le token caché

PISTES DE SOLUTIONS

- React Context combinée à des hooks
- CORS

PLAN D'ACTIONS

- DMC / ER
- Étudier le fonctionnement du CORS
- Se renseigner sur les failles de sécurités actuel
- Se renseigner sur les solutions actuel / les bonnes pratiques
- Sécuriser le frontend

DÉFINITION : MOTS CLÉS

CORS

Le **CORS** (*Cross-Origin Resource Sharing*, « partage des ressources entre origines ») est un **mécanisme de sécurité du navigateur** qui contrôle si une page web servie par une **origine** (couple **protocole + domaine + port**) a le droit d'appeler une API hébergée sur une **origine différente**. Par défaut, la **politique du même origine** (*Same-Origin Policy*) **bloque** ces requêtes croisées pour protéger l'utilisateur. Le serveur autorise explicitement le front-end en renvoyant des **en-têtes HTTP** dédiés :

- `Access-Control-Allow-Origin` : l'origine (ex. `https://app.breezy.fr`) autorisée à consommer l'API ;
- `Access-Control-Allow-Methods` : les méthodes permises (`GET`, `POST`...) ;
- `Access-Control-Allow-Headers` : les en-têtes acceptés, notamment `Authorization` qui porte le **JWT**.

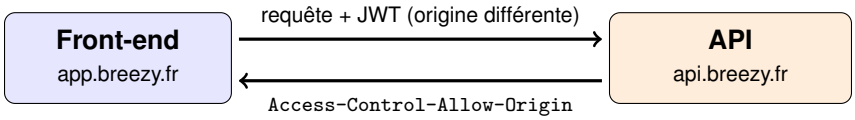


Figure : sans en-tête CORS adéquat, le navigateur bloque l'appel inter-origines. Dans le projet, le front-end React et l'API étant sur des origines distinctes, une **configuration CORS correcte** (côté serveur) est indispensable pour que le **JWT** puisse être transmis ; c'est l'une des contraintes explicites de l'étude de cas.

React Context

Le **React Context** est le mécanisme natif de React permettant de **partager une donnée** (utilisateur connecté, thème, **jeton JWT**) avec un **ensemble de composants** sans la faire transiter manuellement de parent en enfant via les props. Il résout le problème du « **prop drilling** » : faire descendre une même valeur à travers de nombreux niveaux intermédiaires qui n'en ont pas l'usage. Un contexte se crée avec `createContext()`, se diffuse via un **Provider** et se consomme avec le hook `useContext()`.

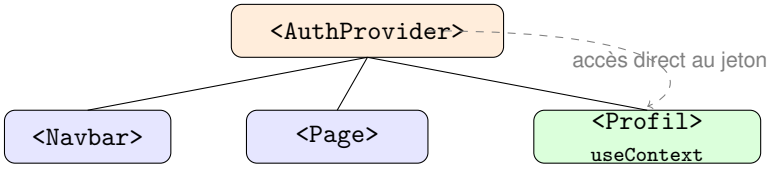


Figure : le contexte rend le JWT disponible à tout l'arbre sans « prop drilling ». C'est la **piste de solution** retenue (combinée aux hooks) pour exposer proprement le jeton aux composants qui en ont besoin.

Provider

Un **Provider** (« fournisseur ») est le composant, livré par chaque contexte (`Contexte.Provider`), qui **englobe une portion de l'arbre** de composants et lui **fournit une valeur**. Tout composant placé **à l'intérieur** du Provider peut lire cette valeur, quel que soit son niveau de profondeur. Dans le projet, un **AuthProvider** centralise l'**état d'authentification** (le JWT, l'utilisateur courant et les fonctions `login`/`logout`) et l'expose à toute l'application :

```
import { createContext, useState } from 'react';

export const AuthContext = createContext(null);   // création du contexte

// Provider : englobe l'app et fournit le jeton + les actions
export function AuthProvider({ children }) {
  const [token, setToken] = useState(null);       // état partagé (le JWT)

  const login  = (jwt) => setToken(jwt);          // stocke le jeton
  const logout = ()    => setToken(null);         // efface le jeton

  return (
    <AuthContext.Provider value={{ token, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

On enveloppe alors l'application (`<AuthProvider><App/></AuthProvider>`) pour que tous les composants accèdent au jeton. Le Provider est le pivot de l'**access store** décrit ci-dessous.

Access store

Un **access store** (« magasin d'accès ») est l'**état centralisé** qui détient les informations d'**authentification et d'accès** de l'utilisateur — au premier rang desquelles le **jeton JWT** — et les met à disposition de toute l'application de façon **contrôlée**. Plutôt que de disperser le jeton dans plusieurs composants, on le **regroupe en un point unique** (ici un contexte React via l'`AuthProvider`, ou une bibliothèque dédiée comme Redux/Zustand). Le store expose typiquement :

- la **valeur du jeton** et l'utilisateur courant (**source de vérité unique**) ;
- les **actions** `login` / `logout` qui le mettent à jour ;
- un indicateur **« est authentifié ? »** dérivé du jeton.

C'est la réponse directe à l'AAV « Employer un access store » : centraliser le jeton facilite sa **sécurisation** (un seul endroit à protéger), son **rafraîchissement** et son **effacement** à la déconnexion, ce qui répond à la problématique « comment sécuriser son token côté frontend ? ».

Hooks

Les **hooks** sont des **fonctions spéciales** de React (préfixées par `use`) qui permettent à un **composant fonctionnel** de « se brancher » sur des fonctionnalités avancées : **état local**, **cycle de vie**, **contexte**, etc. Ils répondent à l'AAV « Interagir avec les composants (hooks) ». Les principaux :

- `useState` : ajoute un **état local** au composant (ex. le chargement) ;
- `useEffect` : exécute des **effets de bord** (appels API) selon le cycle de vie ;
- `useContext` : **consomme un contexte** — ici, lire le **JWT** fourni par l'`AuthProvider` ;
- **hook personnalisé** (`useAuth`) : encapsule une logique réutilisable.

On crée souvent un **hook personnalisé** `useAuth` qui masque les détails du contexte et offre une **interface simple** aux composants :

```
import { useContext } from 'react';
import { AuthContext } from './AuthProvider';

// Hook personnalisé : accès simple et réutilisable au jeton
export function useAuth() {
  return useContext(AuthContext);                // { token, login, logout }
}

// Dans un composant : on lit le JWT sans connaître le contexte
function BoutonDeconnexion() {
  const { logout } = useAuth();
  return <button onClick={logout}>Se déconnecter</button>;
}
```

Combinés au **React Context**, les hooks constituent la **piste de solution** retenue pour exposer et manipuler le jeton de façon réutilisable dans tout le front-end.

Loadbalancing

Le **loadbalancing** (**répartition de charge**) consiste à **distribuer les requêtes entrantes** sur **plusieurs instances** d'un même service afin d'**absorber la charge**, d'**améliorer la disponibilité** (si une instance tombe, les autres prennent le relais) et d'assurer le **routage** vers le bon service. Un **répartiteur de charge** (*load balancer*, ex. **Nginx**) se place en **point d'entrée** et oriente chaque requête selon une stratégie (*round-robin*, moindre charge. . .).

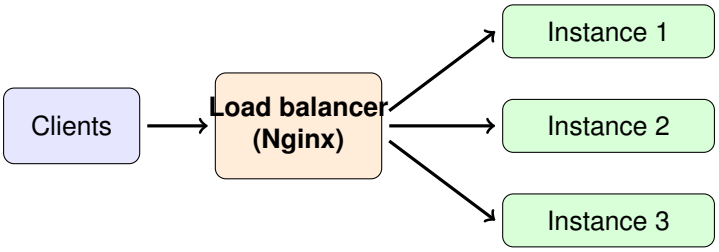


Figure : le répartiteur distribue les requêtes sur plusieurs instances et route vers le bon service. C'est la réponse à l'AAV « Utiliser le système de loadbalancing pour réaliser le

routage » : au-delà de la performance, le même **Nginx** sert de **point d'entrée** qui route les appels (et peut valider le **JWT**) vers les services en aval.

ANALYSE ET RÉSOLUTION : SÉCURITÉ

Rappel de la problématique : « Comment sécuriser son token côté frontend ? »

Le back-end de Yanis délivre déjà des **JWT** ; le risque s'est donc déplacé **côté navigateur**, où le jeton doit être **stocké**, **transporté** et **exposé aux composants** sans jamais être dérobé. La réponse se construit en cinq étapes : **(1)** le choix du **lieu de stockage** du jeton (et les menaces **XSS** / **CSRF**), **(2)** sa **centralisation** dans un **access store** (React Context + Provider), **(3)** son **exposition** via des **hooks** et des **composants réutilisables**, **(4)** son **transport sécurisé** vers l'API (en-tête `Authorization` et **CORS**), et **(5)** le **routage** par **loadbalancing** en point d'entrée. Une synthèse argumentée clôt l'analyse.

1. OÙ ET COMMENT STOCKER LE JETON ? (MENACES XSS / CSRF)

Sécuriser le jeton commence par choisir **où le conserver**. Trois options s'offrent au front-end, avec des compromis de sécurité distincts :

Emplacement	Exposé au XSS	Exposé au CSRF
`localStorage`	**oui** (lisible en JS)	non
Cookie `httpOnly`	**non** (illisible en JS)	oui (à mitiger)
Mémoire (variable JS)	limité	non

Deux menaces structurent le choix : le **XSS** (*Cross-Site Scripting*), où un script injecté **lit le jeton**, et le **CSRF** (*Cross-Site Request Forgery*), où une requête est **émise à l'insu** de l'utilisateur. La meilleure protection consiste à confier le jeton à un **cookie** `httpOnly` (inaccessible au JavaScript, donc immunisé contre le vol par XSS), complété par l'attribut `SameSite=Strict` et un jeton anti-CSRF pour neutraliser le CSRF. Lorsqu'un cookie n'est pas envisageable, on conserve le jeton **en mémoire** dans l'**access store** (perdu au rechargement, mais jamais persisté en clair), en **durcissant le front contre le XSS** (échappement systématique, **CSP**). On évite `localStorage` pour un jeton sensible, précisément parce qu'il est lisible par tout script.

2. CENTRALISER LE JETON DANS UN ACCESS STORE

Plutôt que de disperser le jeton, on le **centralise en un point unique** — un **access store** bâti sur **React Context** et exposé par un **Provider**. Ce store devient la **source de vérité** de l'authentification (jeton, utilisateur, actions `login`/`logout`) et n'écrit **jamais** le jeton en clair dans un emplacement vulnérable :

```jsx
import { createContext, useState, useContext } from 'react';

const AuthContext = createContext(null);

// Access store : le JWT vit EN MÉMOIRE (pas dans localStorage)
export function AuthProvider({ children }) {
  const [token, setToken] = useState(null);       // source de vérité unique

  const login  = (jwt) => setToken(jwt);          // après /login back-end
  const logout = ()    => setToken(null);         // efface le jeton

  const value = { token, login, logout, estAuth: Boolean(token) };
  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export const useAuth = () => useContext(AuthContext);   // accès réutilisable
```

En enveloppant l'application (`<AuthProvider><App/></AuthProvider>`), **un seul endroit** détient le jeton : il est plus simple à **protéger**, à **rafraîchir** et à **effacer** à la déconnexion. C'est la réponse à l'AAV « Employer un access store ».

3. EXPOSER LE JETON : HOOKS ET COMPOSANTS RÉUTILISABLES

Le store est consommé via le **hook** `useAuth`, qui masque les détails du contexte et offre une **interface simple** aux composants (AAV « Interagir avec les composants (hooks) »). On en dérive des **composants réutilisables** qui encapsulent la logique de sécurité une fois pour toutes — par exemple une **route protégée** (AAV « Développer des composants réutilisables ») :

```
import { Navigate } from 'react-router-dom';
import { useAuth } from './AuthProvider';

// Composant réutilisable : protège n'importe quelle page
export function RouteProtegee({ children }) {
  const { estAuth } = useAuth();                    // lit le store via le hook
  return estAuth ? children : <Navigate to="/login" replace />;
}

// Usage : <RouteProtegee><Profil /></RouteProtegee>
```

Ce composant, **indépendant et réutilisable**, applique le même contrôle à toutes les pages sensibles. **Point de sécurité essentiel** : ce garde-fou relève de l'**expérience utilisateur** (ne pas afficher de pages inutiles) et reste **contournable** ; la **protection réelle est toujours imposée côté back-end** par la validation du JWT.

4. TRANSPORTER LE JETON VERS L'API (AUTHORIZATION + CORS)

Le jeton est **joint automatiquement** à chaque requête sortante par un **intercepteur Axios** qui le lit dans le store et le place dans l'en-tête `Authorization: Bearer <token>` :

```
// Intercepteur : ajoute le JWT du store à chaque requête
api.interceptors.request.use((config) => {
  const { token } = store.getState().auth;          // jeton centralisé
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Comme le front (ex. `app.breezy.fr`) et l'API (ex. `api.breezy.fr`) sont sur des **origines différentes**, le navigateur applique la **politique du même origine** et **bloquerait** l'appel sans une **configuration CORS** explicite côté serveur. Le back-end doit donc **autoriser l'origine du front** et l'en-tête `Authorization` :

```
// Configuration CORS côté API (Express)
app.use(cors({
  origin: 'https://app.breezy.fr',          // seule origine autorisée
  methods: ['GET', 'POST', 'PUT', 'DELETE'],
  allowedHeaders: ['Authorization', 'Content-Type'],
```

```
  credentials: true,                    // si cookies httpOnly
}));
```

On évite ainsi le $*$ permissif : **seule l'origine du front** est acceptée, ce qui répond à la contrainte **CORS** du projet tout en laissant passer le **JWT**.

5. ROUTAGE PAR LOADBALANCING EN POINT D'ENTRÉE

En amont des services, un **répartiteur de charge Nginx** sert de **point d'entrée unique** : il **route** les requêtes vers les bonnes instances, répartit la charge et peut **valider le JWT** avant de transmettre. Le front ne connaît ainsi qu'une seule adresse, et la sécurité est appliquée **au plus tôt** :

```
upstream api_backend {            # plusieurs instances -> répartition
  server api1:4000;
  server api2:4000;
}
server {
  listen 80;
  location /api/ {
    proxy_pass http://api_backend;          # routage + load balancing
    proxy_set_header Authorization $http_authorization;  # transmet le JWT
  }
}
```

C'est la réponse à l'AAV « Utiliser le système de loadbalancing pour réaliser le routage » : un même point d'entrée assure **routage**, **disponibilité** et transmission contrôlée du jeton.

6. ARCHITECTURE RETENUE (SCHÉMA ARGUMENTÉ)

L'utilisateur s'authentifie, reçoit un **JWT** qui est **centralisé dans l'access store** ; les **composants** le consomment via le **hook** `useAuth`, l'**intercepteur** le joint à chaque appel, et le **load balancer Nginx** route la requête vers l'API après franchissement du **CORS**.

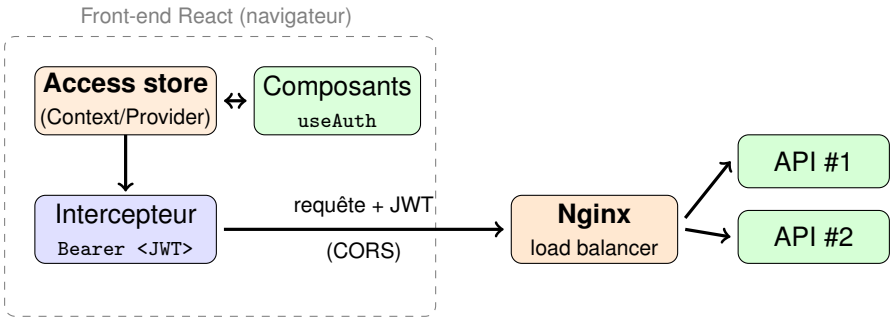


Figure : architecture retenue — jeton centralisé dans l'access store, consommé par les composants via hooks, transporté en `Bearer` à travers le CORS et routé par le load balancer Nginx.

Justification : chaque exigence est satisfaite — **jeton non exposé** (mémoire / cookie `httpOnly`, jamais `localStorage`), **centralisation** dans un access store unique, **exposition réutilisable** par hooks et composants, **transport contrôlé** via `Authorization` et une **CORS** restrictive, et **routage** par load balancer en point d'entrée.

CONCLUSION

En réponse à la problématique, sécuriser le jeton côté front-end ne tient pas à une mesure unique mais à une **chaîne de décisions cohérentes**. Le premier choix est celui du **stockage** : on écarte `localStorage` (lisible par un XSS) au profit d'un **cookie** `httpOnly` ou, à défaut, d'une conservation **en mémoire** dans l'access store, en durcissant le front contre le **XSS** et le **CSRF**.

Le jeton est ensuite **centralisé** dans un **access store** (React Context + Provider), **source de vérité unique** bien plus simple à protéger, rafraîchir et effacer. Il est **exposé** aux composants par le **hook** `useAuth` et par des **composants réutilisables** (route protégée), étant entendu que ce contrôle front relève de l'**UX** : la **protection réelle reste imposée côté back-end**.

Enfin, le jeton est **transporté** en `Authorization: Bearer` via un intercepteur, à travers une **CORS restrictive** (seule l'origine du front autorisée), puis **routé** par un **load balancer Nginx** en point d'entrée unique. L'ensemble constitue un **modèle reproductible** de sécurisation du jeton côté client, applicable à toutes les interfaces de l'entreprise.

CAPITALISATION

AAV	Commentaire	Statut
Développer des composants réutilisables	Composant `RouteProtegee` indépendant et réutilisable, encapsulant le contrôle d'accès front et applicable à toute page sensible.	Acquis
Employer un access store	Jeton centralisé dans un access store (React Context + Provider) servant de source de vérité unique (`token`, `login`, `logout`), illustré par du code commenté.	Acquis
Interagir avec les composants (hooks)	Hooks React (`useState`, `useContext`) et hook personnalisé `useAuth` exposant le jeton aux composants de façon simple et réutilisable.	Acquis
Utiliser (Modéliser) JWT pour le processus d'authentification	Processus modélisé : jeton délivré au `login`, stocké sans exposition (cookie `httpOnly`/mémoire vs `localStorage`), joint en `Bearer` et validé côté back-end.	Acquis
Utiliser le système de load-balancing pour réaliser le routage	Load balancer Nginx en point d'entrée unique répartissant la charge sur plusieurs instances et routant les requêtes (avec transmission du JWT), illustré par une configuration.	Acquis

RESSOURCES

- **JWT (introduction & débogueur)** : https://jwt.io/introduction
- **MDN – CORS (Cross-Origin Resource Sharing)** : https://developer.mozilla.org/fr/docs/Web/HTTP/CORS
- **React – Context** : https://react.dev/learn/passing-data-deeply-with-context
- **React – Hooks (useContext)** : https://react.dev/reference/react/useContext
- **OWASP – JWT & stockage côté client (Cheat Sheet)** : https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- **OWASP – Cross-Site Scripting (XSS)** : https://owasp.org/www-community/attacks/xss/
- **OWASP – Cross-Site Request Forgery (CSRF)** : https://owasp.org/www-community/attacks/csrf
- **Documentation Axios (intercepteurs)** : https://axios-http.com/fr/docs/interceptors
- **Nginx – répartition de charge (load balancing)** : https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/